

Planejamento de Infraestrutura de Pagamentos — Kalles PDV

Contexto do Projeto Atual

O Kalles já possui uma base muito bem estruturada para pagamentos:

- **Frontend** (`payment-panel.tsx`): UI com 4 métodos — `CASH`, `PIX`, `CREDIT_CARD`, `DEBIT_CARD`. Já envia o método e valor via `POST /api/sales/{token}/payments` e exibe troco.
- **Backend** (`PaymentStrategy` + `PaymentFactory`): Padrão Strategy com 4 implementações — todas ainda **simuladas** (retornam UUID aleatório e `confirmed = true` imediatamente).
- **Entidade `Payment`**: Já possui `method`, `amount`, `changeAmount`, `transactionId`, `confirmed`. A estrutura de dados está pronta.

O desafio real está em substituir as simulações por integrações reais, cada uma com sua própria complexidade.

Mapa de Complexidade por Método

Método	Complexidade	Integração necessária	Confirmação
Dinheiro (CASH)	★ Baixa	Nenhuma (local)	Imediata
PIX	★ ★ Média	API do Banco Central (via PSP)	Assíncrona via webhook
Cartão Débito	★ ★ ★ Alta	TEF / maquininha (SDK)	Síncrona via TEF
Cartão Crédito	★ ★ ★ Alta	TEF / maquininha (SDK)	Síncrona via TEF

1. Dinheiro (CASH) — Baixa Complexidade ☒ Quase pronto

Como funciona

O sistema registra o valor recebido, calcula o troco e armazena localmente. **Não há integração externa.**

O que já está feito

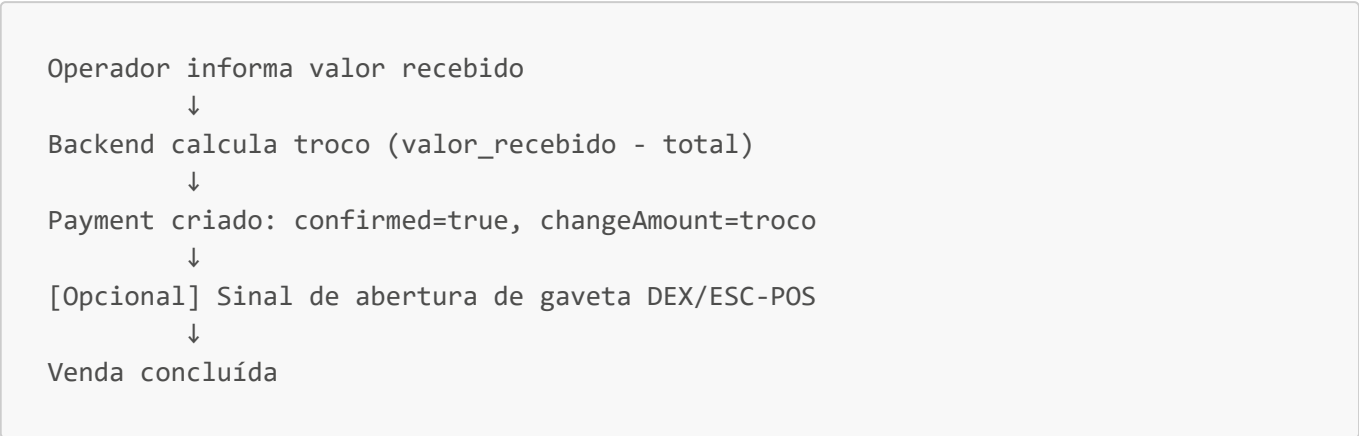
- `CashPaymentStrategy` cria o `Payment` com `confirmed = true` imediatamente ☒
- Cálculo de `changeAmount` no `Sale.java` ☒
- Exibição do troco no `payment-panel.tsx` ☒

O que ainda falta

- **Gaveta de dinheiro física**: Opcional. Se o PDV tiver uma gaveta ESC/POS (ex: Epson, Bematech), o backend pode enviar um comando de abertura via porta serial ou rede local após confirmar o pagamento em dinheiro.
 - Biblioteca sugerida: `JPOS` (Java) ou chamar script externo.

- **Sangria / suprimento:** Controle de quanto dinheiro entrou/saiu do caixa ao longo da sessão (já existe `initialAmount` na sessão, mas não há movimentações intermediárias).

Diagrama de Fluxo

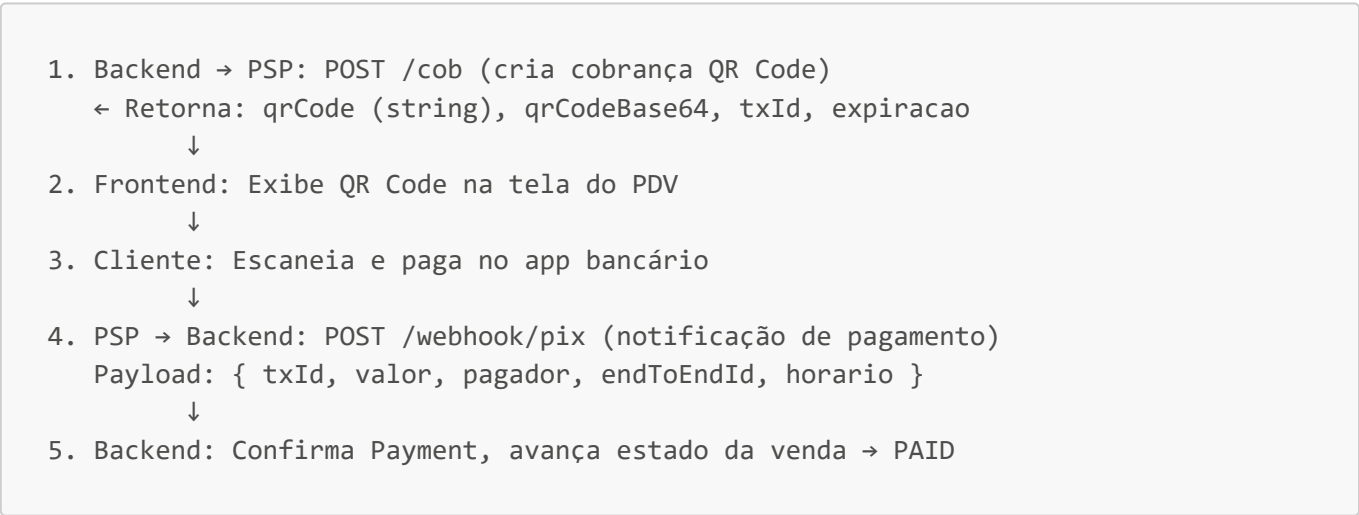


2. PIX — Média Complexidade ◇

Como funciona

O sistema gera uma **cobrança PIX** via API do seu PSP (Provedor de Serviço de Pagamento). O cliente escaneia o QR Code. O PSP recebe a confirmação do Banco Central e notifica o seu sistema via **webhook**.

Fluxo obrigatório



Providers recomendados (Brasil)

Provider	Custo	Dificuldade de integração	Observação
Mercado Pago	~1,3% por transação	★ Mais fácil	SDK Java oficial, sandbox gratuito
EFI Bank (ex-Gerencianet)	Taxa menor para volume	★★ Média	API PIX nativa, muito usada em PDVs

Provider	Custo	Dificuldade de integração	Observação
Asaas	~1%	★ ★ Média	Boa documentação
API direta BACEN	Gratuita	★ ★ ★ ★ Muito difícil	Requer certificado ICP-Brasil e aprovação como PSP

[!TIP] Para começar, use **Mercado Pago** (sandbox gratuito e SDK Java). Quando tiver volume, migre para EFI Bank ou similar para reduzir tarifas.

O que precisa ser implementado

Backend:

1. **Configuração PSP:** Chaves de API (`application.yml`), certificado mTLS se usar API BACEN direta.
2. **PixPaymentStrategy** (substituir simulação):
 - Chamar `POST /v2/cob` no PSP → retornar `txId` e QR Code
 - Salvar `Payment` com `confirmed = false, transactionId = txId`
3. **Endpoint de Polling ou WebSocket** para o frontend saber quando o pagamento confirmou:
 - `GET /api/sales/{token}/payments/{paymentId}/status`
4. **Webhook handler:** `POST /webhook/pix`
 - Verificar assinatura HMAC do PSP
 - Buscar `Payment` por `transactionId`
 - Marcar `confirmed = true` e avançar estado da venda
5. **Expiração:** PIX tem prazo (ex: 5 minutos). Implementar tarefa agendada para cancelar cobranças expiradas.

Frontend:

1. Ao selecionar PIX, chamar `addPayment` e exibir QR Code retornado.
2. Fazer polling em `GET .../status` a cada 2s enquanto `confirmed = false`.
3. Quando confirmado, avançar automaticamente para o estado pago.

Dados adicionais no `PaymentRequest`

```
// Para PIX, o response precisa incluir:
interface PixPaymentResponse extends PaymentResponse {
    qrCode: string;           // string EMV para copiar-colar
    qrCodeBase64: string;    // imagem para exibir
    expiresAt: string;       // ISO 8601
}
```

3. Cartão de Crédito e Débito — Alta Complexidade

Modelo de integração: TEF vs. Gateway Online

São dois cenários completamente diferentes dependendo de como o PDV funciona:

Cenário A: PDV com maquininha física (TEF — Transferência Eletrônica de Fundos)

Este é o modelo de **PDV presencial de loja física**. A maquininha (POS/Pinpad) é conectada ao computador do caixa e o sistema de PDV se comunica com ela.

```
Software PDV (Kalles)
  ↓ comando de pagamento (valor, tipo)
Biblioteca TEF (local, roda no caixa)
  ↓ protocolo proprietário
Maquininha / Pinpad (Stone, Cielo, Rede, Getnet)
  ↓ resposta (aprovado/negado, NSU, código autorização)
Backend Kalles
  ↓ registra Payment confirmado
```

Bibliotecas TEF para Java:

- **SiTef (Software Express)**: Líder no Brasil, integra com todas as adquirentes.
- **CLiSiTef**: Versão client do SiTef, SDK via DLL/SO.
- **Stone Payments SDK**: SDK próprio da Stone para TEF.
- **PayGo / Elgin**: Alternativas mais acessíveis para pequenos negócios.

[!WARNING] A integração TEF exige instalação de software local (daemon/serviço) no computador do caixa, além de homologação com a adquirente. É um processo mais longo (semanas a meses).

Cenário B: E-commerce ou sem maquininha física (Gateway online)

Se o Kalles vier a ter vendas online ou aceitar cartão sem maquininha física:

- **Mercado Pago (Checkout Pro / Transparent)**: Mais fácil.
- **Stripe**: Excelente DX, mas cobra em USD.
- **PagSeguro**: Amplamente usado no Brasil.

Fluxo TEF (Cenário A — mais provável para um PDV)

```
1. PDV Kalles chama endpoint do backend: POST /api/sales/{token}/payments
   { method: "CREDIT_CARD", amount: 150.00 }
   ↓
2. Backend delega para CreditCardPaymentStrategy
   → Envia requisição para serviço TEF local (via HTTP ou socket local)
   ↓
3. Serviço TEF aciona maquininha física
   → Cliente insere/aproxima cartão, digita senha
   ↓
4. Maquininha retorna: NSU, código de autorização, bandeira, parcelas
   ↓
5. Backend registra Payment:
   confirmed = true
   transactionId = NSU + código autorização
```



6. Frontend avança para estado PAID

Dados adicionais necessários na entidade **Payment**

Para cartão, há campos importantes que o modelo atual não suporta:

```
// Campos a adicionar na entidade Payment
@Column(length = 10)
private String cardBrand;           // "VISA", "MASTERCARD", "ELO" etc.

@Column(length = 20)
private String authorizationCode;  // código de autorização do banco emissor

@Column(length = 20)
private String nsu;                // Número Sequencial Único (NSU)

@Column
private Integer installments;      // parcelas (crédito)

@Column(length = 10)
private String acquirer;           // "STONE", "CIELO", "REDE" etc.
```

Parcelas (crédito)

Crédito parcelado adiciona mais uma camada:

- O PDV precisa perguntar "quantas parcelas?" ao operador.
- Os juros podem ser por conta do cliente (com acréscimo no valor) ou absorvidos pela loja.
- A adquirente retorna o valor de cada parcela e o custo total.

4. Decisões de Arquitetura que Você Precisa Tomar

[!IMPORTANT] Estas decisões impactam diretamente o roadmap de implementação:

4.1 Contrato entre PDV e maquininha

- **TEF local** (SiTef, Stone, PayGo): Maquininha conectada fisicamente ao caixa via USB/serial. O PDV precisa rodar um cliente TEF local. Mais complexo, mas é o padrão do varejo físico.
- **Maquininha independente** (sem integração TEF): O operador passa o cartão manualmente e só registra o pagamento no sistema depois. Simples de implementar, mas perde automação.

4.2 Provedor PIX

- Qual banco/PSP você vai usar? Isso define as credenciais, variáveis de ambiente e o formato da API.

4.3 Ambiente de desenvolvimento

- Todos os providers têm **sandbox/homologação**. Você vai precisar de contas de teste antes de ir para produção.

4.4 Gestão de webhooks em desenvolvimento local

- Para receber webhooks do PSP localmente, use **ngrok** ou **Cloudflare Tunnel** para expor o `localhost:8080` para a internet durante o desenvolvimento.

5. Roadmap Sugerido (Fases)

Fase 1 — Dinheiro (já pronto + pequenos ajustes)

- ☐ Revisar cálculo de troco no backend
- ☐ (Opcional) Integrar abertura de gaveta de dinheiro
- ☐ Testes de fluxo completo

Fase 2 — PIX (melhor custo-benefício para começar)

- ☐ Criar conta de desenvolvedor no Mercado Pago
- ☐ Configurar credenciais no `application.yml` (separar por perfil: dev/prod)
- ☐ Implementar `PixPaymentStrategy` real
- ☐ Criar `PixWebhookController` com verificação de assinatura
- ☐ Criar endpoint de polling de status do pagamento
- ☐ Atualizar frontend para exibir QR Code e fazer polling
- ☐ Testes em sandbox

Fase 3 — Cartão (TEF ou registro manual)

- **Opção rápida:** Modo "manual" — operador informa NSU após passar na maquininha
- **Opção completa:** Integração TEF com SiTef ou Stone
- ☐ Definir qual adquirente/maquinhinha será usada
- ☐ Implementar estratégia de acordo com a escolha
- ☐ Adicionar campos de `cardBrand`, `nsu`, `authorizationCode` na entidade
- ☐ Atualizar UI para campos de parcelas (crédito)

6. Segurança e Boas Práticas

Tópico	Recomendação
Credenciais de PSP	Nunca no código. Use variáveis de ambiente (<code>PAYMENT_SECRET_KEY=...</code>)
Validação de webhooks	Sempre verificar assinatura HMAC/JWT do provider antes de processar
Idempotência	Usar <code>txId</code> único para evitar processar o mesmo webhook duas vezes
HTTPS obrigatório	Webhooks de PSPs só funcionam em HTTPS em produção

Tópico	Recomendação
PCI-DSS	Nunca trafegar dados de cartão (número, CVV) pelo seu backend. Isso é responsabilidade do TEF/gateway
Logs	Registrar pagamentos com <code>txId</code> , horário e status mas sem dados sensíveis de cartão

7. Resumo do que Mudar no Código

Backend (`kalles-sale`)

Arquivo	Mudança
<code>Payment.java</code>	Adicionar <code>cardBrand</code> , <code>nsu</code> , <code>authorizationCode</code> , <code>installments</code> , <code>acquirer</code>
<code>PixPaymentStrategy.java</code>	Substituir simulação por chamada real ao PSP
<code>CreditCardPaymentStrategy.java</code>	Integrar com TEF ou modo manual
<code>DebitCardPaymentStrategy.java</code>	Idem crédito
<code>PaymentResult.java</code>	Adicionar campo <code>qrCode</code> , <code>qrCodeBase64</code> para PIX
<code>PixWebhookController.java</code>	[NOVO] Recebe e processa webhooks do PSP
<code>application.yml</code>	Adicionar configurações dos providers por perfil

Frontend (`kalles/frontend`)

Arquivo	Mudança
<code>payment-panel.tsx</code>	Exibir QR Code PIX + polling de status
<code>types/index.ts</code>	Adicionar <code>PixPaymentResponse</code> com <code>qrCode</code> , <code>expiresAt</code>
<code>sale.service.ts</code>	Adicionar <code>getPaymentStatus(token, paymentId)</code>
<code>payment-panel.tsx</code>	Para crédito, adicionar seletor de parcelas